

MarginRankingLoss#

<https://pytorch.org/docs/stable/generated/torch.nn.MarginRankingLoss.html>

Description:

Creates a criterion that measures the loss given inputs x_1 (1D mini-batch or 0D Tensor), x_2 (1D mini-batch or 0D Tensor), and a label y (1D mini-batch or 0D Tensor) (containing 1 or -1). If $y=1$ then it is assumed the first input should be ranked higher (have a larger value) than the second input, and vice-versa for $y=-1$. The loss function for each pair of samples in the mini-batch is: $\max(0, -\text{margin} + x_1 - x_2)$ for $y=1$ and $\max(0, \text{margin} + x_1 - x_2)$ for $y=-1$. **margin** (float, optional) – Has a default value of 0.0. **size_average** (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field **size_average** is set to False, the losses are instead summed for each minibatch. Ignored when reduce is False. Default: True

Function Signature

```
class torch.nn.MarginRankingLoss(margin=0.0,  
size_average=None, reduce=None, reduction='mean')[source]#
```

Parameters

Shape:

```
forward(input1, input2, target)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> loss = nn.MarginRankingLoss()  
>>> input1 = torch.randn(3, requires_grad=True)  
>>> input2 = torch.randn(3, requires_grad=True)  
>>> target = torch.randn(3).sign()  
>>> output = loss(input1, input2, target)  
>>> output.backward()
```

Detailed Documentation

Documentation

Creates a criterion that measures the loss given inputs $x_1 \times x_1 \times x_1$, $x_2 \times x_2 \times x_2$, two 1D mini-batch or 0D Tensors, and a label 1D mini-batch or 0D Tensor y (containing 1 or -1).

If $y=1$ then it assumed the first input should be ranked higher (have a larger value) than the second input, and vice-versa for $y=-1$.

The loss function for each pair of samples in the mini-batch is:

`margin` (float, optional) – Has a default value of 0.0.

`size_average` (bool, optional) – Deprecated (see `reduction`). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to `False`, the losses are instead summed for each minibatch. Ignored when `reduce` is `False`. Default: `True`

`reduce` (bool, optional) – Deprecated (see `reduction`). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`. When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`

`reduction` (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: `size_average` and `reduce` are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`. Default: 'mean'

Input1: $(N)(N)(N)$ or $()()$ where N is the batch size.

Input2: $(N)(N)(N)$ or $()()$, same shape as the Input1.

Target: $(N)(N)(N)$ or $()()()$, same shape as the inputs.

Output: scalar. If reduction is 'none' and Input size is not $()()()$, then $(N)(N)(N)$.

Runs the forward pass.